

Part 13

ORM in Django Part-3

VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

ORM Relationship

A relationship, in the context of databases, is a situation that exists between two relational database tables when one table has a foreign key that references the primary key of the other table. Relationships allow relational databases to split and store data in different tables, while linking disparate data items.

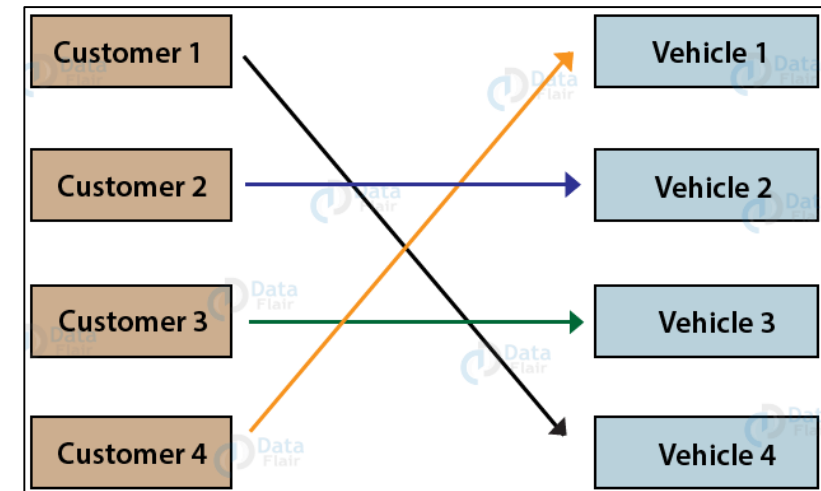
Django ORM provides a level of abstraction which makes it easy to work with objects. ORM will automatically relate the object's attributes to corresponding table fields. We are going to make some models and check their field relationships in other tables.

One-To-One Relationship

A one-to-one relationship exists between two tables. For each row in table1, there shall be a row/ entity in table2. Let's understand it with an example.

There are two tables here, Customer and Vehicle. Every customer owns only one vehicle. Thus, a one-to-one relationship exists. Now, paste this code in models.py to implement this.

OneToOne



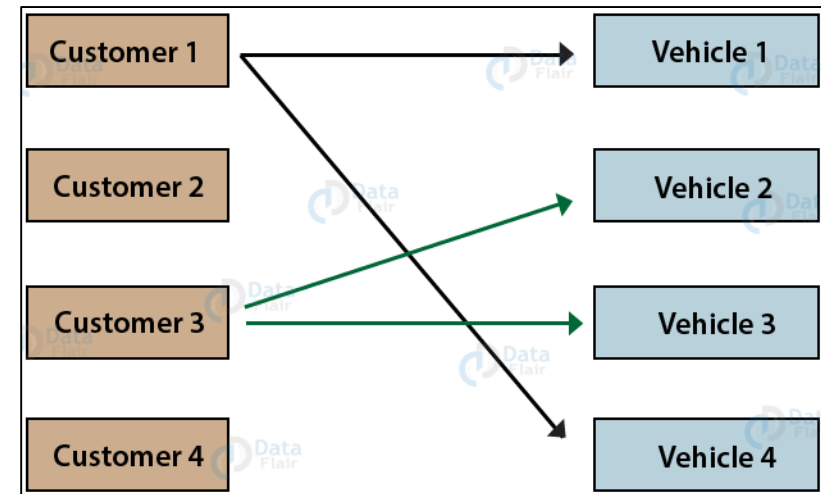
One-To-Many Relationship

A one to many relationships is where one object from table1 can have multiple relations with entities in table2. Although, table2 objects will have only one relation to the object of table1.

Just like one to one we can have one-to-many relationships. We are going to use a different model for showcasing this.

Let's understand this from the example:

ForeignKey

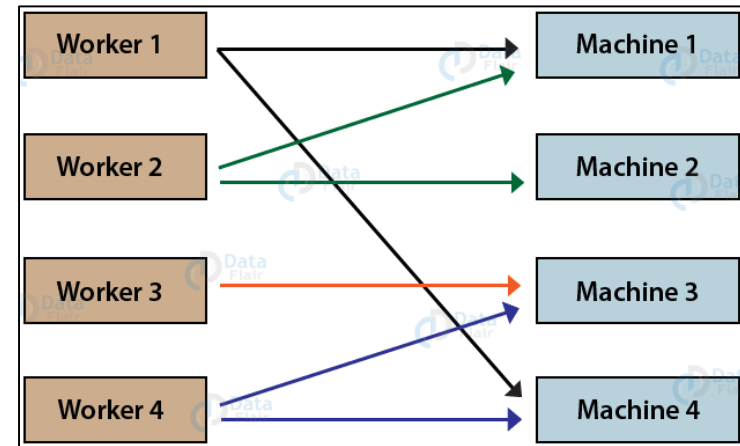


Many-To-Many Relationship

We can also implement many to many relations in the database. In this case, we are taking examples of multiple drivers.

Here we have multiple workers for multiple machines. A worker can be assigned to operate more than one machine. Also, a machine can be operated by multiple workers one at a time.

ManyToMany



Many-To-Many Relationship

- ✓ This is a special case of relationships. Many to Many Relationship requires a separate table. There they store objects related to other objects.
- ✓ These relationships are common for objects and classes. These can be difficult to write with SQL.

Summary

Django ORM and other ORMs are very powerful tools for web-developers. However, there are some issues which cannot be solved by them. ORMs are abstractions and automate a process. There are points where they can generate more complex queries than manual.

It all depends on the developer and their skill with the corresponding database. On the contrary, the ORM exists only for SQL and RDBMS (Relational database management systems). There are no ORMs for NoSQL databases. So, it depends on your use case and how you perceive the data.

Summary

ORMs have both bad and good sides. Particularly Django ORM is highly favored by web-developers. It's not very efficient when compared to SQLAlchemy. The deep integration with the Django framework is where it really shines. Also, Django ORM is an abstraction layer. It provides us with features like Querysets and migrations and these relations.

Example

Thanks for Reading